LearnWithAditya

FULL JAVA NOTES

FOR BEGINNERS • CHAPTER 1 TO 5

UML • EXCEPTION HANDLING • CLASSES & OBJECTS

INHERITANCE & POLYMORPHISM • MULTITHREADING & PACKAGES

Definitions • Concepts • Java Code • Examples • Tricks • Exam Questions

Powered by

Aditya Pratap Singh

LEARN • PRACTICE • GROW

CONTENTS — CHAPTER-WISE ORDER

Chapter 1 — UML (Unified Modeling Language)

Chapter 2 — Exception Handling

Chapter 3 — Classes and Objects

Chapter 4 — Inheritance and Polymorphism

Chapter 5 — Multithreading and Packages

Final Revision — Quick tricks and important questions

CHAPTER 1 — UML (UNIFIED MODELING LANGUAGE)

1.1 What is UML?

1.2 Main categories of UML diagrams

1.3 Use-case diagram

Example — Student Notes Website

Text sketch: [Student] — (View Notes); [Student] — (Download PDF); [Admin] — (Upload Notes).

1.4 Class diagram

Example class sketch

```
Student
----------------------
- studentId : int
- name : String
----------------------
```

```
+ displayDetails() : void
```

1.5 Sequence and activity diagrams

1.6 UML advantages and limitations

Exam questions — Chapter 1

CHAPTER 2 — EXCEPTION HANDLING

2.1 Definition and purpose

2.2 Types of problems

2.3 try, catch and finally

```
public class DivideDemo {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        } finally {
            System.out.println("Program finished");
        }
    }
}
```

Output: Cannot divide by zero; Program finished

2.4 throw vs throws

```
static void checkAge(int age) {
    if (age < 18) {
        throw new IllegalArgumentException("Age must be 18+");
    }
}
```

2.5 Custom exception

```
class InvalidMarksException extends Exception {
    InvalidMarksException(String message) {
        super(message);
    }
}

static void validateMarks(int marks) throws InvalidMarksException {
    if (marks < 0 || marks > 100) {
        throw new InvalidMarksException("Marks must be 0 to 100");
    }
}
```

2.6 try-with-resources

```
try (java.util.Scanner sc = new java.util.Scanner(System.in)) {
    int n = sc.nextInt();
    System.out.println(n);
}
```

Memory tricks — Chapter 2

Exam questions — Chapter 2

CHAPTER 3 — CLASSES AND OBJECTS

3.1 Class and object

```
class Student {
    int rollNo;
    String name;

    void display() {
        System.out.println(rollNo + " " + name);
    }
}

Student s = new Student();
s.rollNo = 101;
s.name = "Aditya";
s.display();
```

3.2 Constructor

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
Student s = new Student("Aditya", 20);
```

3.3 this keyword

```
Student(String name) {
    this.name = name;
}
```

3.4 Methods and method overloading

```
int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }
```

3.5 static keyword

```
class Counter {
    static int total = 0;
    Counter() { total++; }
}
Counter c1 = new Counter();
Counter c2 = new Counter();
System.out.println(Counter.total); // 2
```

3.6 Access modifiers and encapsulation

```
class Account {
    private double balance;
    public double getBalance() { return balance; }
    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
}
```

3.7 Object creation and garbage collection

Memory tricks — Chapter 3

Exam questions — Chapter 3

CHAPTER 4 — INHERITANCE AND POLYMORPHISM

4.1 Inheritance

Types commonly described in exams

```
class Animal {
    void eat() { System.out.println("Eating"); }
}
class Dog extends Animal {
    void bark() { System.out.println("Barking"); }
}
Dog d = new Dog();
d.eat();
d.bark();
```

4.2 super keyword

```
class Person {
    Person(String name) { System.out.println(name); }
}
class Student extends Person {
    Student() { super("Aditya"); }
}
```

4.3 Polymorphism

```
class Animal {
    void sound() { System.out.println("Animal sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); }
}
Animal a = new Dog();
a.sound(); // Bark
```

4.4 Method overriding rules

4.5 Abstract class and interface

```
interface Printable {
    void print();
}
class Report implements Printable {
    public void print() { System.out.println("Printing report"); }
}
```

4.6 Composition vs inheritance

Memory tricks — Chapter 4

Exam questions — Chapter 4

CHAPTER 5 — MULTITHREADING AND PACKAGES

5.1 Process and thread

5.2 Creating threads

```
class MyTask implements Runnable {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(i);
        }
    }
}
public class ThreadDemo {
    public static void main(String[] args) {
        Thread t = new Thread(new MyTask());
        t.start();
    }
}
```

Key point: start() requests a new thread of execution; calling run() directly is just a normal method call.

5.3 Thread life cycle

5.4 Important thread methods

5.5 Synchronization, race condition and deadlock

```
class Counter {
    private int count = 0;
    synchronized void increment() { count++; }
    synchronized int getCount() { return count; }
}
```

5.6 Packages

```
package college.notes;

public class StudentNotes {
    public void show() {
        System.out.println("Java notes");
    }
}
```

5.7 Access and package tips

Memory tricks — Chapter 5

Exam questions — Chapter 5

FINAL QUICK REVISION & EXAM TRICKS

One-line definitions

Common mistakes to avoid

How to structure a 7-mark answer

Practice coding checklist

LearnWithAditya • Powered by Aditya Pratap Singh